

Decison_Tree_Regressor

August 13, 2025

1 Decision Tree Classifier Practical Implementation

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
[2]: from sklearn.datasets import load_diabetes
data = load_diabetes()
print(data['DESCR'])
```

.. _diabetes_dataset:

Diabetes dataset

Ten baseline variables, age, sex, body mass index, average blood pressure, and six blood serum measurements were obtained for each of n = 442 diabetes patients, as well as the response of interest, a quantitative measure of disease progression one year after baseline.

****Data Set Characteristics:****

:Number of Instances: 442

:Number of Attributes: First 10 columns are numeric predictive values

:Target: Column 11 is a quantitative measure of disease progression one year after baseline

:Attribute Information:

- age age in years
- sex
- bmi body mass index
- bp average blood pressure

- s1 tc, total serum cholesterol
- s2 ldl, low-density lipoproteins
- s3 hdl, high-density lipoproteins
- s4 tch, total cholesterol / HDL
- s5 ltg, possibly log of serum triglycerides level
- s6 glu, blood sugar level

Note: Each of these 10 feature variables have been mean centered and scaled by the standard deviation times the square root of `n\_samples` (i.e. the sum of squares of each column totals 1).

Source URL:

<https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html>

For more information see:

Bradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani (2004) "Least Angle Regression," Annals of Statistics (with discussion), 407-499.

(https://web.stanford.edu/~hastie/Papers/LARS/LeastAngle_2002.pdf)

[3]: data

```
[3]: {'data': array([[ 0.03807591,  0.05068012,  0.06169621, ..., -0.00259226,
                    0.01990749, -0.01764613],
                  [-0.00188202, -0.04464164, -0.05147406, ..., -0.03949338,
                    -0.06833155, -0.09220405],
                  [ 0.08529891,  0.05068012,  0.04445121, ..., -0.00259226,
                    0.00286131, -0.02593034],
                  ...,
                  [ 0.04170844,  0.05068012, -0.01590626, ..., -0.01107952,
                    -0.04688253,  0.01549073],
                  [-0.04547248, -0.04464164,  0.03906215, ...,  0.02655962,
                    0.04452873, -0.02593034],
                  [-0.04547248, -0.04464164, -0.0730303 , ..., -0.03949338,
                    -0.00422151,  0.00306441]]),
      'target': array([151.,  75., 141., 206., 135.,  97., 138.,  63., 110., 310.,
                    101.,
                    69., 179., 185., 118., 171., 166., 144.,  97., 168.,  68.,  49.,
                    68., 245., 184., 202., 137.,  85., 131., 283., 129.,  59., 341.,
                    87.,  65., 102., 265., 276., 252.,  90., 100.,  55.,  61.,  92.,
                    259.,  53., 190., 142.,  75., 142., 155., 225.,  59., 104., 182.,
                    128.,  52.,  37., 170., 170.,  61., 144.,  52., 128.,  71., 163.,
                    150.,  97., 160., 178.,  48., 270., 202., 111.,  85.,  42., 170.,
                    200., 252., 113., 143.,  51.,  52., 210.,  65., 141.,  55., 134.,
                    42., 111.,  98., 164.,  48.,  96.,  90., 162., 150., 279.,  92.,
                    83., 128., 102., 302., 198.,  95.,  53., 134., 144., 232.,  81.,
                    104.,  59., 246., 297., 258., 229., 275., 281., 179., 200., 200.,
```

```

173., 180., 84., 121., 161., 99., 109., 115., 268., 274., 158.,
107., 83., 103., 272., 85., 280., 336., 281., 118., 317., 235.,
60., 174., 259., 178., 128., 96., 126., 288., 88., 292., 71.,
197., 186., 25., 84., 96., 195., 53., 217., 172., 131., 214.,
59., 70., 220., 268., 152., 47., 74., 295., 101., 151., 127.,
237., 225., 81., 151., 107., 64., 138., 185., 265., 101., 137.,
143., 141., 79., 292., 178., 91., 116., 86., 122., 72., 129.,
142., 90., 158., 39., 196., 222., 277., 99., 196., 202., 155.,
77., 191., 70., 73., 49., 65., 263., 248., 296., 214., 185.,
78., 93., 252., 150., 77., 208., 77., 108., 160., 53., 220.,
154., 259., 90., 246., 124., 67., 72., 257., 262., 275., 177.,
71., 47., 187., 125., 78., 51., 258., 215., 303., 243., 91.,
150., 310., 153., 346., 63., 89., 50., 39., 103., 308., 116.,
145., 74., 45., 115., 264., 87., 202., 127., 182., 241., 66.,
94., 283., 64., 102., 200., 265., 94., 230., 181., 156., 233.,
60., 219., 80., 68., 332., 248., 84., 200., 55., 85., 89.,
31., 129., 83., 275., 65., 198., 236., 253., 124., 44., 172.,
114., 142., 109., 180., 144., 163., 147., 97., 220., 190., 109.,
191., 122., 230., 242., 248., 249., 192., 131., 237., 78., 135.,
244., 199., 270., 164., 72., 96., 306., 91., 214., 95., 216.,
263., 178., 113., 200., 139., 139., 88., 148., 88., 243., 71.,
77., 109., 272., 60., 54., 221., 90., 311., 281., 182., 321.,
58., 262., 206., 233., 242., 123., 167., 63., 197., 71., 168.,
140., 217., 121., 235., 245., 40., 52., 104., 132., 88., 69.,
219., 72., 201., 110., 51., 277., 63., 118., 69., 273., 258.,
43., 198., 242., 232., 175., 93., 168., 275., 293., 281., 72.,
140., 189., 181., 209., 136., 261., 113., 131., 174., 257., 55.,
84., 42., 146., 212., 233., 91., 111., 152., 120., 67., 310.,
94., 183., 66., 173., 72., 49., 64., 48., 178., 104., 132.,
220., 57.]),

```

'frame': None,

'DESCR': '.. _diabetes_dataset:\n\nDiabetes dataset\n-----\n\nTen
baseline variables, age, sex, body mass index, average blood\npressure, and six
blood serum measurements were obtained for each of n =\n442 diabetes patients,
as well as the response of interest, a\nquantitative measure of disease
progression one year after baseline.\n\n**Data Set Characteristics:**\n\n: Number
of Instances: 442\n\n: Number of Attributes: First 10 columns are numeric
predictive values\n\n: Target: Column 11 is a quantitative measure of disease
progression one year after baseline\n\n: Attribute Information:\n - age
age in years\n - sex\n - bmi body mass index\n - bp average
blood pressure\n - s1 tc, total serum cholesterol\n - s2 ldl,
low-density lipoproteins\n - s3 hdl, high-density lipoproteins\n - s4 tch, total cholesterol / HDL\n - s5 ltg, possibly log of serum
triglycerides level\n - s6 glu, blood sugar level\n\nNote: Each of these
10 feature variables have been mean centered and scaled by the standard
deviation times the square root of `n\_samples` (i.e. the sum of squares of each
column totals 1).\n\nSource

URL:\n<https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html>\n\nFor more information see:\nBradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani (2004) "Least Angle Regression," Annals of Statistics (with discussion), 407-499.\n(https://web.stanford.edu/~hastie/Papers/LARS/LeastAngle_2002.pdf)\n',
 'feature_names': ['age',
 'sex',
 'bmi',
 'bp',
 's1',
 's2',
 's3',
 's4',
 's5',
 's6'],
 'data_filename': 'diabetes_data_raw.csv.gz',
 'target_filename': 'diabetes_target.csv.gz',
 'data_module': 'sklearn.datasets.data'}

```
[4]: df = pd.DataFrame(data.data, columns=['age',  

  'sex',  

  'bmi',  

  'bp',  

  's1',  

  's2',  

  's3',  

  's4',  

  's5',  

  's6'])
```

```
[5]: df.head()
```

```
[5]:
```

	age	sex	bmi	bp	s1	s2	s3	\
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	

	s4	s5	s6
0	-0.002592	0.019907	-0.017646
1	-0.039493	-0.068332	-0.092204
2	-0.002592	0.002861	-0.025930
3	0.034309	0.022688	-0.009362
4	-0.002592	-0.031988	-0.046641

```
[6]: X = df
      y = data['target']
```

```
[7]: from sklearn.model_selection import train_test_split
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      ↪ random_state=42)
```

```
[8]: X_train.head()
```

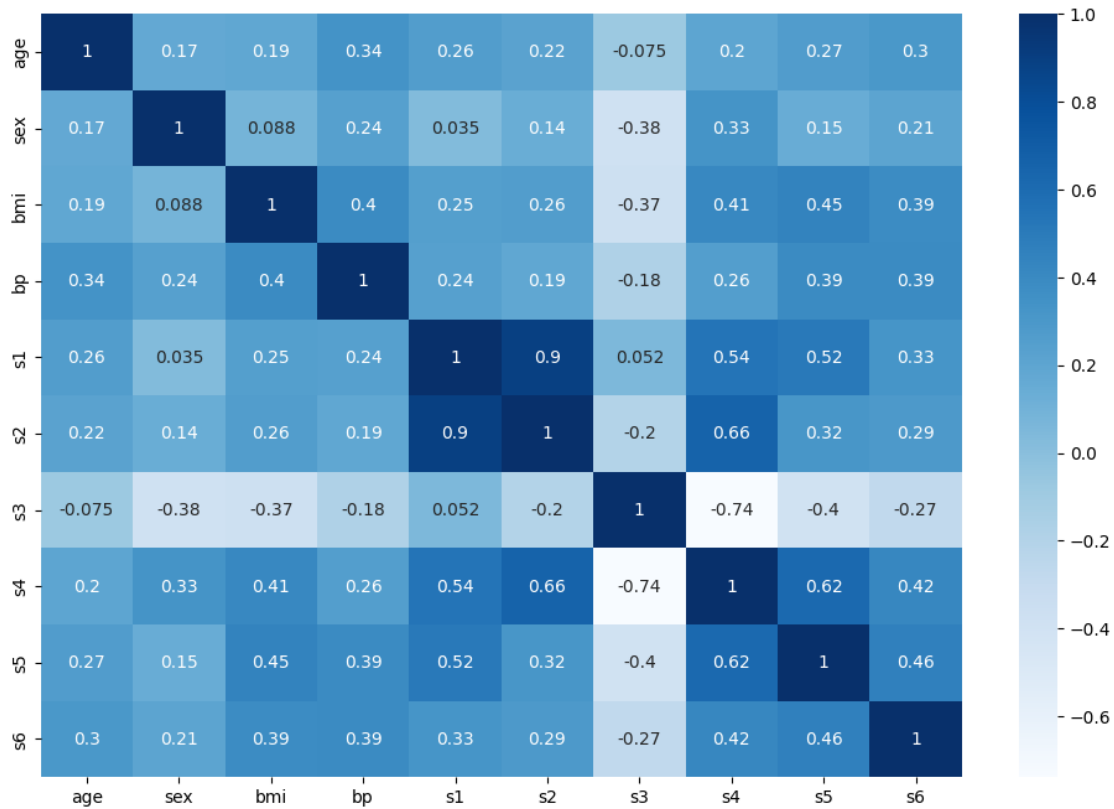
```
[8]:
```

	age	sex	bmi	bp	s1	s2	s3	\
17	0.070769	0.050680	0.012117	0.056301	0.034206	0.049416	-0.039719	
66	-0.009147	0.050680	-0.018062	-0.033213	-0.020832	0.012152	-0.072854	
137	0.005383	-0.044642	0.049840	0.097615	-0.015328	-0.016345	-0.006584	
245	-0.027310	-0.044642	-0.035307	-0.029770	-0.056607	-0.058620	0.030232	
31	-0.023677	-0.044642	-0.065486	-0.081413	-0.038720	-0.053610	0.059685	

	s4	s5	s6
17	0.034309	0.027364	-0.001078
66	0.071210	0.000272	0.019633
137	-0.002592	0.017036	-0.013504
245	-0.039493	-0.049872	-0.129483
31	-0.076395	-0.037129	-0.042499

```
[9]: plt.figure(figsize=(12,8))
      sns.heatmap(df.corr(), annot=True, cmap='Blues')
```

```
[9]: <Axes: >
```



```
[10]: from sklearn.tree import DecisionTreeRegressor
reg = DecisionTreeRegressor()
```

2 Hyperparameter Tuning

```
[11]: params = {
    'criterion':['squared_error', 'friedman_mse', 'absolute_error'],
    'splitter':['best', 'random'],
    'max_depth':[1,2,3,4,5,10,15,20,25],
    'min_samples_split':[1,2,3],
    'min_samples_leaf':[1,2,3],
    'max_features':['auto', 'sqrt', 'log2']
}
```

```
[12]: from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(estimator=reg, param_grid=params,
    cv=5, scoring='neg_mean_squared_error')
grid.fit(X_train, y_train)
```

```
[12]: GridSearchCV(cv=5, estimator=DecisionTreeRegressor(),
                param_grid={'criterion': ['squared_error', 'friedman_mse',
                                           'absolute_error'],
                            'max_depth': [1, 2, 3, 4, 5, 10, 15, 20, 25],
                            'max_features': ['auto', 'sqrt', 'log2'],
                            'min_samples_leaf': [1, 2, 3],
                            'min_samples_split': [1, 2, 3],
                            'splitter': ['best', 'random']},
                scoring='neg_mean_squared_error')
```

```
[13]: grid.best_estimator_
```

```
[13]: DecisionTreeRegressor(max_depth=2, max_features='log2', min_samples_split=3)
```

```
[14]: grid.best_params_
```

```
[14]: {'criterion': 'squared_error',
      'max_depth': 2,
      'max_features': 'log2',
      'min_samples_leaf': 1,
      'min_samples_split': 3,
      'splitter': 'best'}
```

```
[15]: y_pred = grid.predict(X_test)
```

```
[16]: from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
      print(r2_score(y_test, y_pred))
      print(mean_absolute_error(y_test, y_pred))
      print(mean_squared_error(y_test, y_pred))
```

```
0.11281489584455384
```

```
57.11202783342512
```

```
4700.441310839061
```